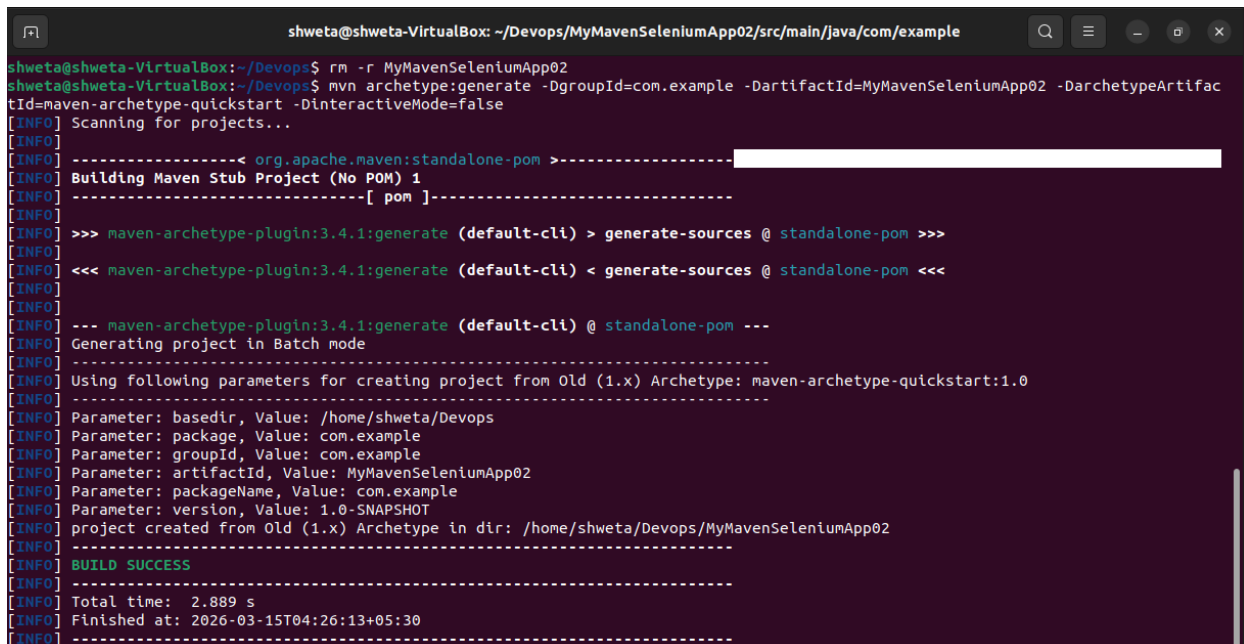


Program 4: Maven Project(2) To Connect To Web Servers Using Selenium Dependency

Step 1: Create a maven project on terminal

```
mvn archetype:generate -DgroupId=com.example -DartifactId=
MyMavenSeleniumApp02 -DarchetypeArtifactId=maven-archetype-
quickstart -DinteractiveMode=false
```



```
shweta@shweta-VirtualBox: ~/Devops/MyMavenSeleniumApp02/src/main/java/com/example
shweta@shweta-VirtualBox:~/Devops$ rm -r MyMavenSeleniumApp02
shweta@shweta-VirtualBox:~/Devops$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp02 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
[INFO] Parameter: basedir, Value: /home/shweta/Devops
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenSeleniumApp02
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/shweta/Devops/MyMavenSeleniumApp02
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 2.889 s
[INFO] Finished at: 2026-03-15T04:26:13+05:30
[INFO]
[INFO]
```

Step 2: Edit the pom.xml file

Add the Selenium dependency by going to the Maven Central Repository, searching for Selenium, selecting **Selenium Java**, choosing the latest version, and copying the dependency code. Paste this dependency into the pom.xml file.

```
<project
xmlns="http://maven.apache.org/POM/4.0.0
"
xmlns:xsi="http://www.w3.org/2001/XMLSchema
chema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>

  <artifactId>MyMavenSeleniumApp02</artifactId>

  <packaging>jar</packaging>

  <version>1.0-SNAPSHOT</version>

  <name>MyMavenSeleniumApp02</name>

  <url>http://maven.apache.org</url>

  <dependencies>

    <dependency>

      <groupId>junit</groupId>

      <artifactId>junit</artifactId>

      <version>3.8.1</version>

      <scope>test</scope>

    </dependency>
```

```
<!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->

<dependency>

  <groupId>org.seleniumhq.selenium</groupId>

  <artifactId>selenium-java</artifactId>

  <version>4.29.0</version>

</dependency>

</dependencies>

<!-- Build: Configuring plugins and build settings -->

<build>

<plugins>

<!-- Example: Maven Compiler Plugin to compile Java code -->

<plugin>

  <groupId>org.apache.maven.plugins</groupId>

  <artifactId>maven-compiler-plugin</artifactId>

  <version>3.8.1</version>

  <configuration>

    <source>1.7</source>

    <target>1.7</target>

  </configuration>
```

```
</plugin>

<!-- Example: Maven Surefire Plugin to run tests -->

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-surefire-plugin</artifactId>

<version>2.22.2</version>

</plugin>

<plugin>

    <groupId>org.apache.maven.plugins</groupId>

    <artifactId>maven-jar-plugin</artifactId>

    <version>3.0.1</version>

    <configuration>

        <archive>

<manifestEntries>

            <Main-Class>com.example.App</Main-Class>

</manifestEntries>

        </archive>

    </configuration>

</plugin> </plugins>
</build> </project>
```

3: Before writing the App.java file, inspect the Practice Test Automation webpage to find the element IDs required for automation.

Right-click on the webpage and select **Inspect**. The HTML code will be displayed. Click on the cursor icon (arrow symbol) available on the top-left of the inspect panel, then hover over the username field to identify its ID, which is **username**.

Similarly, check the password field (ID: **password**) and the submit button (ID: **submit**). Make a note of these IDs, as they will be used in the App.java file.

App.java for selenium project

```
package com.example;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
public class App
{
    public static void main( String[] args )
    {
        WebDriver driver=new ChromeDriver();
        driver.get("https://practicetestautomation.com/practice-test-login/");
        driver.manage().window().maximize();
    }
}
```

```

driver.findElement(By.id("username")).sendKeys("student");

driver.findElement(By.id("password")).sendKeys("Password123");

driver.findElement(By.id("submit")).click();

}}

```

Step 4: build and run the maven application

mvn clean install

```

shweta@shweta-VirtualBox: ~/Devops/MyMavenSeleniumApp02
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/shweta/Devops/MyMavenSeleniumApp02/target/classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenSeleniumApp02 ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/shweta/Devops/MyMavenSeleniumApp02/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenSeleniumApp02 ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/shweta/Devops/MyMavenSeleniumApp02/target/test-classes
[INFO]
[INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ MyMavenSeleniumApp02 ---
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.05 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- maven-jar-plugin:3.0.1:jar (default-jar) @ MyMavenSeleniumApp02 ---
[INFO] Building jar: /home/shweta/Devops/MyMavenSeleniumApp02/target/MyMavenSeleniumApp02-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MyMavenSeleniumApp02 ---
[INFO] Installing /home/shweta/Devops/MyMavenSeleniumApp02/target/MyMavenSeleniumApp02-1.0-SNAPSHOT.jar to /home/shweta/.m2/repository/com/example/MyMavenSeleniumApp02/1.0-SNAPSHOT/MyMavenSeleniumApp02-1.0-SNAPSHOT.jar
[INFO] Installing /home/shweta/Devops/MyMavenSeleniumApp02/pom.xml to /home/shweta/.m2/repository/com/example/MyMavenSeleniumApp02/1.0-SNAPSHOT/MyMavenSeleniumApp02-1.0-SNAPSHOT.pom
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 25.037 s
[INFO] Finished at: 2026-03-15T04:41:44+05:30
[INFO]

```

Step 5: run the below command

mvn exec:java -Dexec.mainClass="com.example.App"

```

[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 26.846 s
[INFO] Finished at: 2026-03-17T05:03:02Z
[INFO] -----

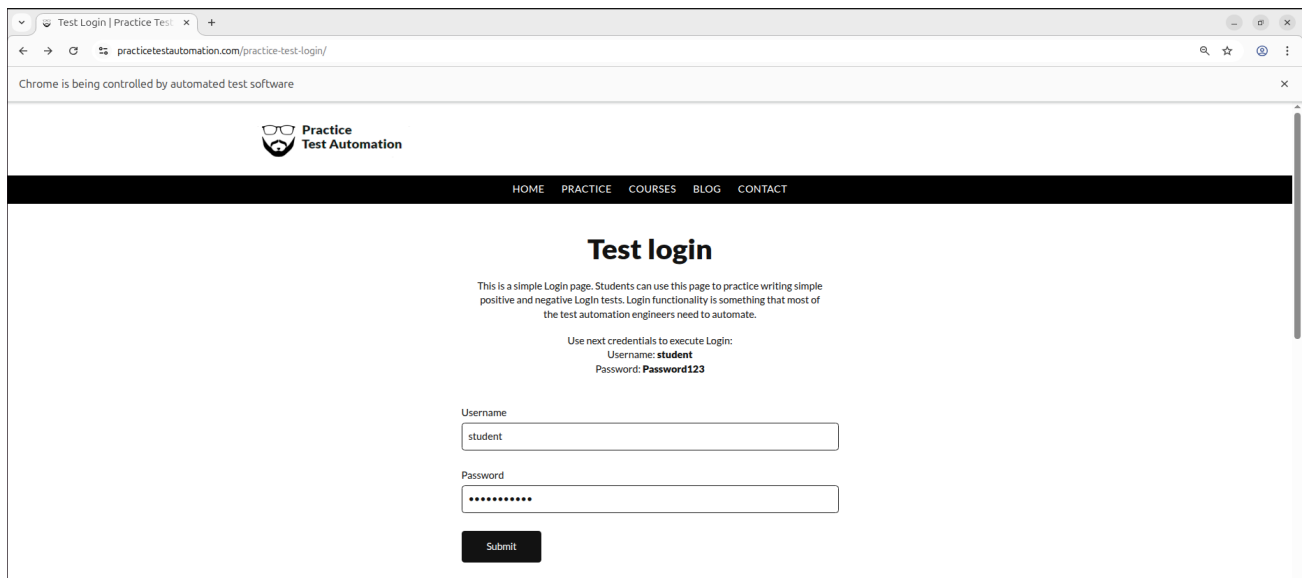
```

Practice Test Automation Website

<https://practicetestautomation.com/practice-test-login/> is a sample website used for testing and learning web automation tools like Selenium.

It is used for:

- Writing automation scripts and UI tests
- Practicing login functionality testing
- Learning and improving test automation skills



Pushing MyMavenSeleniumApp02 to Github:

```
shweta@shweta-VirtualBox: ~/Devops/MyMavenSeleniumApp02
shweta@shweta-VirtualBox:~$ cd Devops
shweta@shweta-VirtualBox:~/Devops$ cd MyMavenSeleniumApp02
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp02$ git remote add origin https://github.com/Shweta311204/MyMavenSeleniumApp02.git
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp02$ git remote set-url origin https://ghp_iQyXF5wI04shwetshweshweshweshwetss
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp02$ git push -u origin master
Enumerating objects: 39, done.
Counting objects: 100% (39/39), done.
Delta compression using up to 3 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (39/39), 8.50 KiB | 543.00 KiB/s, done.
Total 39 (delta 0), reused 0 (delta 0), pack-reused 0
remote: This repository moved. Please use the new location:
remote:   https://github.com/Shweta311204/MyMavenSeleniumApp02.git
To https://github.com/Shweta311204/MyMavenSeleniumApp02.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
```

Setting Up Jenkins Pipeline for MyMavenSeleniumApp02

Step 1: To deploy **MyMavenSeleniumApp02**, open the terminal, cd into the **MyMavenSeleniumApp02** folder, create a **Jenkinsfile** containing the pipeline code, and then commit and push it to GitHub.

```
pipeline {  
  
    agent any // Use any available agent tools {  
  
        maven 'Maven' // Ensure this matches the name configured in Jenkins  
    }  
  
    stages {  
  
        stage('Checkout') { steps {  
  
            git branch: 'master', url:  
            'https://github.com/Shweta311204/MyMavenSeleniumApp02.git'  
        }  
    }  
  
        stage('Build') { steps {  
  
            sh 'mvn clean package' // Run Maven build  
        }  
    }  
  
        stage('Test') { steps {  
  
            sh 'mvn test' // Run unit tests  
        }  
    }  
  
        stage('Run Application') { steps {  
  
            // Start the JAR application
```



```

sh 'mvn exec:java -Dexec.mainClass="com.example.App"'
}
}
}

post {

success {

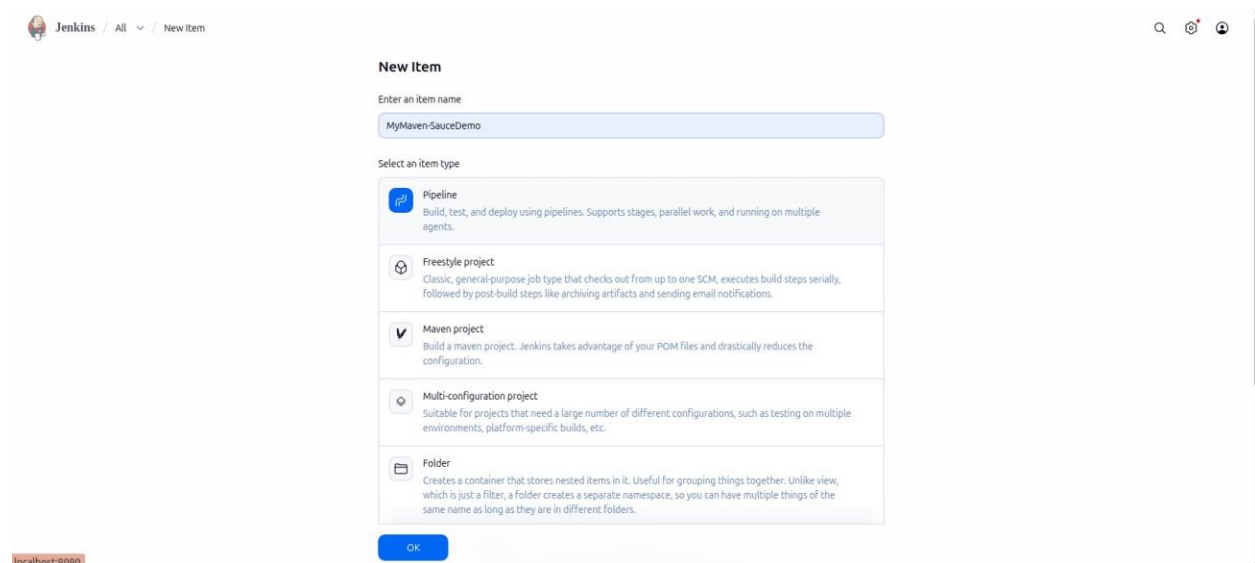
echo 'Build and deployment successful!'
}

failure {

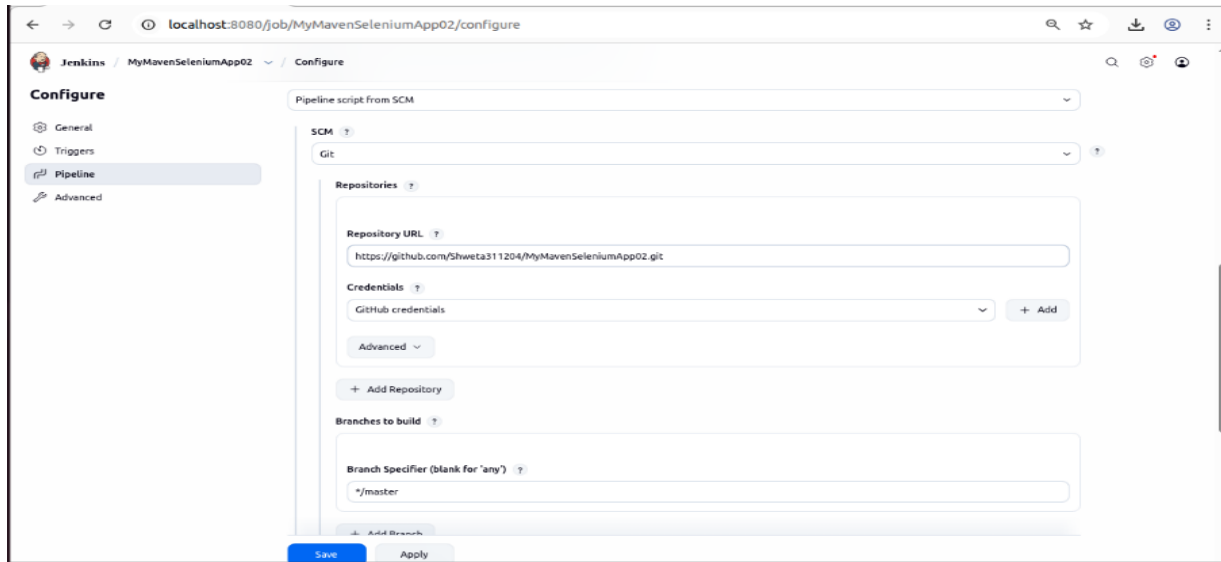
echo 'Build failed!'
}
}
}

```

Step 2: To deploy **MyMavenSeleniumApp02**, you first need to create a Jenkins pipeline job that will use the Jenkinsfile from your project repository. Then, go to the Jenkins home page and click on “**New Item**” to create a job, enter the item name (e.g., *MyMavenSeleniumApp02*), and select **Pipeline** as the project type. This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click “**OK**” to proceed to the job configuration page.



Step 3: In **MyMavenSeleniumApp02**, go to the **Pipeline** section and under **Definition**, select **Pipeline script from SCM**. Choose **Git** as the SCM, and under **Repository URL**, paste the GitHub URL of **MyMavenSeleniumApp02**. In the **Credentials** field, select the previously created credential (e.g., *GitHub Credential*). Finally, in **Script Path**, enter the name of the Jenkinsfile you created in the **MyMavenSeleniumApp01** folder and pushed to GitHub. Then click **Apply** and **Save** to finalize the configuration.



Step 4: After clicking **Apply** and **Save**, go back to **MyMavenSeleniumApp02** in **Jenkins** and click **Build Now**. If you get an error like:

java.lang.NoClassDefFoundError:
io/github/bonigarcia/wdm/WebDriverManager

this error occurs when the application is unable to find required external libraries (such as Selenium WebDriver and WebDriverManager) at runtime. Even though the project builds successfully, the dependencies are not included inside the final JAR file, so Jenkins fails while executing the program.

To solve this issue, instead of running the application using a JAR file, we use Maven execution so that all dependencies defined in pom.xml are properly loaded during runtime execution.

This ensures that Selenium and WebDriverManager libraries are available when the program runs in Jenkins.

Replace the existing execution method with the following:

```
mvn exec:java -Dexec.mainClass="com.example.App"
```

Jenkins Pipeline Update:

```
stage('Run Application') { steps {  
  
  sh 'mvn exec:java -Dexec.mainClass="com.example.App"'  
}  
}
```

This stage is used to execute the Java application directly through Maven so that all dependencies defined in the project are included during runtime.

pom.xml Dependencies (Required Fix):

```
<dependency>  
  
<groupId>org.seleniumhq.selenium</groupId>  
  
<artifactId>selenium-java</artifactId>  
  
<version>4.29.0</version>  
  
</dependency>
```

```
<dependency>  
  
<groupId>io.github.bonigarcia</groupId>  
  
<artifactId>webdrivermanager</artifactId>  
  
<version>5.7.0</version>
```

</dependency>

These dependencies are required to enable Selenium automation and automatic browser driver management during execution.

Changes made in App.java to solve the error:

To resolve the **NoClassDefFoundError (WebDriverManager missing at runtime)**, the following changes were ensured in App.java:

Added **WebDriverManager setup** to automatically handle ChromeDriver dependency: `WebDriverManager.chromedriver().setup();`

This line automatically downloads, sets up, and manages the correct version of ChromeDriver required for the installed Chrome browser. It removes the need to manually download or configure the driver path and ensures compatibility between Chrome and ChromeDriver during Jenkins execution.

Configured **ChromeOptions for Jenkins/Linux environment** to avoid browser crash in headless mode:

```
ChromeOptions options = new ChromeOptions(); options.addArguments("--headless=new"); options.addArguments("--no-sandbox"); options.addArguments("--disable-dev-shm-usage"); options.addArguments("--disable-gpu"); options.addArguments("--remote-allow-origins=*");
```

These options ensure that Chrome runs in a headless (no UI) mode suitable for Jenkins/Linux servers and prevent common issues like browser crash, sandbox errors, and memory limitations during execution.

Used **ChromeDriver with options** instead of normal driver: `WebDriver driver = new ChromeDriver(options);`

This ensures that the browser is launched with the configured ChromeOptions settings (such as headless mode and Jenkins-safe arguments), making the automation stable and suitable for execution in CI/CD environments like Jenkins.

Ensured proper **login automation flow** after driver initialization:

```
driver.get("https://www.saucedemo.com/"); driver.findElement(By.id("user-  
name")).sendKeys("standard_user");  
driver.findElement(By.id("password")).sendKeys("secret_sauce");  
driver.findElement(By.id("login-button")).click();
```

This performs the complete login automation flow in the SauceDemo application by opening the website, entering valid credentials, and clicking the login button to validate successful user authentication.

The following are the updated pom.xml, App.java, and Jenkinsfile: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"  
  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
  
        http://maven.apache.org/maven-v4_0_0.xsd">  
  
    <modelVersion>4.0.0</modelVersion>  
  
    <groupId>com.example</groupId>  
  
    <artifactId>MyMavenSeleniumApp02</artifactId>  
  
    <version>1.0-SNAPSHOT</version>  
  
    <packaging>jar</packaging>  
  
    <name>MyMavenSeleniumApp02</name>  
  
    <!-- Java Version -->  
  
    <properties>  
  
        <maven.compiler.source>11</maven.compiler.source>
```

```
<maven.compiler.target>11</maven.compiler.target>

</properties>

<dependencies>

  <!-- JUnit (Test only) -->

  <dependency>

    <groupId>junit</groupId>

    <artifactId>junit</artifactId>

    <version>4.13.2</version>

    <scope>test</scope>

  </dependency>

  <!-- Selenium -->

  <dependency>

    <groupId>org.seleniumhq.selenium</groupId>

    <artifactId>selenium-java</artifactId>

    <version>4.41.0</version>

  </dependency>

</dependencies>

<build>

  <plugins>
```

```
<!-- Compiler Plugin -->

<plugin>

  <groupId>org.apache.maven.plugins</groupId>

  <artifactId>maven-compiler-plugin</artifactId>

  <version>3.11.0</version>

</plugin>

<!-- Surefire Plugin (for tests) -->

<plugin>

  <groupId>org.apache.maven.plugins</groupId>

  <artifactId>maven-surefire-plugin</artifactId>

  <version>3.2.5</version>

</plugin>

<!-- Shade Plugin (creates fat JAR with dependencies) -->

<plugin>

  <groupId>org.apache.maven.plugins</groupId>

  <artifactId>maven-shade-plugin</artifactId>

  <version>3.5.0</version>

  <executions>

    <execution>

      <phase>package</phase>
```

```
<goals>

  <goal>shade</goal>

</goals>

<configuration>

<createDependencyReducedPom>>false</createDependencyReducedPom>

  <transformers>

    <transformer
      complementation="org.apache.maven.plugins.shade.resource.ManifestResourceTr
      ansformer">

      <mainClass>com.example.App</mainClass>

    </transformer>

  </transformers>

</configuration>

</execution>

</executions>

  </plugin>

</plugins>

</build>

</project>
```


App.java

```
package com.example;

import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;

import org.openqa.selenium.chrome.ChromeOptions;

public class App {

    public static void main(String[] args) {

        // Create options FIRST

        ChromeOptions options = new ChromeOptions();

        options.addArguments("--headless=new");

        options.addArguments("--no-sandbox");

        options.addArguments("--disable-dev-shm-usage");

        options.addArguments("--remote-allow-origins=*");

        // Pass options to driver

        WebDriver driver = new ChromeDriver(options);

        driver.get("http://practicetestautomation.com/practice-test-login/");

        driver.findElement(By.id("username")).sendKeys("student");

        driver.findElement(By.id("password")).sendKeys("Password123");

        driver.findElement(By.id("submit")).click();
```

```
        System.out.println("Login successful, Title: " + driver.getTitle());

        driver.quit();

    }

}
```

Jenkinsfile:

```
pipeline {

    agent any // Use any available agent

    tools {

        maven 'Maven' // Ensure this matches the name configured in Jenkins

        jdk 'JDK'

    }

    stages {

        stage('Checkout') {

            steps {

                git branch: 'master', url:
'https://github.com/Shweta311204/MyMavenSeleniumApp02.git'

            }

        }

        stage('Build') {

            steps {

                sh 'mvn clean package' // Run Maven build

            }

        }

    }

}
```

```
    }  
  }  
  
  stage('Test') {  
    steps {  
      sh 'mvn test' // Run unit tests  
    }  
  }  
  
  stage('Run Application') {  
    steps {  
      // Start the JAR application  
      sh 'java -jar target/MyMavenSeleniumApp02-1.0-SNAPSHOT.jar'  
    }  
  }  
}  
  
post {  
  success {  
    echo 'Build and deployment successful!'  
  }  
  
  failure {  
    echo 'Build failed!'
```

```

    }

}
}

```

Step 5: After making the changes, go back to **MyMavenSeleniumApp02** in Jenkins and click **Build Now**. The build should now complete successfully without any errors.

This happens because the dependencies (such as **Selenium** and **WebDriverManager**) are properly included in the project through Maven configuration, and the application is executed using `mvn exec:java`. So during runtime, Jenkins is able to load all required libraries correctly, and issues like **java.lang.NoClassDefFoundError** are resolved.

